

产品角色手册

AI-Native SDLC — PM（产品角色）操作手册

本手册是 AI-Native SDLC 方法体系的 PM 角色培训教材，面向产品经理系统讲解在 AI 原生研发模式下的工作方法、操作步骤和协作规则。学完即可上手实操。

1. 角色定位

1.1 核心职责

在 AI-Native SDLC 中，产品经理不再只是"写需求文档的人"，而是**需求建模师**和**AI 协作编排者**。核心职责包括：

- **需求建模**：将模糊的业务需求结构化拆解为 Epic → Feature → User Story 三级模型
- **Spec 质量把关**：使用 AI 生成需求初稿，但必须进行人工校验，确保产出物达到"AI 可消费"的质量标准
- **跨角色协调**：在篝火评审中串联 设计、研发、测试 各角色，确保需求理解一致
- **Spec 流转管理**：将校验通过的 User Story 流转到 OpenSpec，驱动下游设计、开发和测试

1.2 产出物链路

PM 的核心产出物形成一条从宏观到微观的链路，最终驱动 OpenSpec 变更：

代码块

```
1  PRD (产品全景)
2    └─ Epic (产品/系统级)
3      └─ Feature (功能模块级)
4        └─ User Story (最小可交付用户价值)
5          └─ OpenSpec Change (proposal + design + specs)
```

每一级产出物的存放位置和命名规则如下：

层级	存放位置	命名规则	说明
PRD	pm/sakila-prd.md	—	产品全景，一个项目一份
Epic		Exx 两位编号	

	pm/epic/Exx-name/README.md		业务全景、核心流程、里程碑规划
Feature	pm/epic/Exx-name/Fxx-name/README.md	Fxx 两位编号	独立可交付的功能集
User Story	pm/epic/Exx-name/Fxx-name/USxx-name.md	USxx 两位编号	最小可交付单元，对应 OpenSpec change

1.3 与其他角色的关系

- **PM → UED**: User Story 的「交互说明」字段是 UED 出设计稿的依据
- **PM → Dev**: User Story 的「数据契约」字段是后端 API 设计和前端页面实现的依据
- **PM → QA**: User Story 的「验收标准」（Given/When/Then）是 QA 编写测试用例的依据
- **PM → 全角色**: 通过 OpenSpec 流转，各角色并行消费 specs 生成各自产物

2. 常用命令速查

以下命令在 Claude Code 中直接使用：

场景	命令	说明
需求脑暴	/opsx:explore	进入探索模式，AI 作为思考伙伴帮你梳理需求，只思考不实现
从 User Story 生成 OpenSpec 变更	/opsx:propose <change-name>	生成 proposal + design + specs + tasks（可选）
开发完成后归档变更	/opsx:archive <change-name>	归档 change 到 archive/，聚合 specs 到长期能力库

3. 典型工作路径

3.1 全流程概览

- 1

① 脑暴探索
openspec →

→

② 生成 Epic/Feature/US

→

③ 人工校验

→

④ 流转
openspec →

⑤ 篝火评审 specs →

⑥ 实施 →

⑦ 归档
- 2

/opsx:explore
多角色并行评审

直接对话提示词

开发测试

/opsx:archive

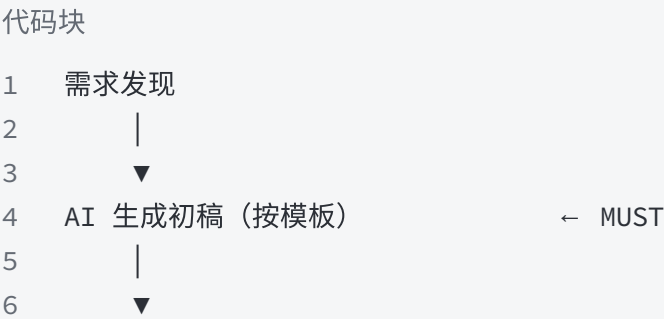
PM 审查

/opsx:propose

3.2 各步骤简要说明

步骤	优先级	做什么	产出物
① 脑暴探索	SHOULD	用 SDD 框架探索需求边界，梳理业务全景	思路整理（不生成文件）
② 生成初稿	MUST	AI 按模板生成结构化 Epic/Feature/User Story 文档	.md 需求文档
③ 人工校验	MUST	PM 逐项审查，检查最小粒度四项标准	校验通过的需求文档
④ 流转 OpenSpec	MUST	将 User Story 转为 OpenSpec change	openspec/changes/xx/ 下的全套文件
⑤ 篝火评审	SHOULD	产品 + 设计 + 开发 + 测试多角色评审 specs	评审意见及修正后的 specs
⑥ 实施	—	开发和测试阶段（非 PM 主责）	代码 + 测试用例
⑦ 归档	MUST	归档变更、聚合 specs 到长期能力库	openspec/specs/ 长期规格

3.3 详细流程图



```
7  PM 人工校验                                ← MUST
8      └─ 不通过 → 修正后重新生成
9      |
10     ▼ 通过
11  流转 to openspec
12      └─ openspec-propose 生成 change
13      └─ change: proposal → specs
14      |
15     ▼
16  篝火评审 specs                            ← SHOULD
17      └─ PM: 确认 specs 与 US 验收标准对齐
18      └─ UED: 交互说明 → 设计稿
19      └─ Backend: 数据契约 → 技术方案
20      └─ Frontend: 交互说明 → 前端实现
21      └─ QA: 验收标准 → 测试用例
22      |
23     ▼ 评审通过
24  实施 (开发 + 测试)
25      |
26     ▼ 开发完成
27  归档变更
28      └─ /opsx:archive 归档 change 到 openspec/changes/archive/
29      └─ 聚合 specs 到 openspec/specs/ (系统的长期能力规格库)
30      |
31     ▼
32  更新 US 状态 → 关闭
```

4. 分步操作指南

4.1 需求脑暴与探索 (SHOULD)

概念说明

在正式生成需求文档之前，先用 SDD (Specification Driven Development) 脑暴框架探索需求边界。这一步的目的是：

- 梳理业务全景，识别遗漏的功能点
- 探索边界场景和隐含需求
- 明确需求的优先级和拆分策略

脑暴阶段**只思考、不实现**——AI 不会写代码或创建文件，但可以读取项目代码和文档辅助分析。

具体操作

方式 A：使用 explore 命令

在 Claude Code 中输入：

```
prompt  
1 /opsx:explore
```

进入探索模式后，直接描述你的想法，AI 会作为思考伙伴帮你梳理需求。

实操提示词示例 — 探索新 Epic：

```
prompt  
1 创建一个Epic，DVD 租赁店后台，员工用来处理日常业务的。需要：影片管理、库存管理、客户管理、租赁业务、门店员工管理。有管理员和普通员工两种角色。帮我梳理一下。
```

实操提示词示例 — 探索新 Feature：

```
prompt  
1 在 @pm/epic/E01-management 创建一个 Feature，用来管理影片，包括影片的增删改查、评级设置、定价、关联演员和分类
```

多轮完善提示词（重复3-5轮）：

```
prompt  
1 /opsx:explore 参考 @pm/epic/E01-management/README.md ，帮我完善此epic
```

```
代码块  
1 /opsx:explore 参考 @pm/epic/E01-management/README.md ，增加xxx方面的内容
```

4.2 AI 生成 Spec 初稿（MUST）

概念说明

脑暴完成后，需要将思路转化为结构化文档。这是 AI-Native SDLC 的核心要求——**必须使用 AI 生成初稿**。AI 会按照预定义的模板（`pm/TEMPLATE.md`）生成符合规范的结构化文档，确保产出物字段完整、格式统一，可被下游角色和 AI 直接消费。

这一步按 Epic → Feature → User Story 的层级逐层生成。

具体操作

生成 Epic 文档：

直接在脑暴完成后，AI 将梳理结果按 Epic 模板输出到指定目录（`pm/CLAUDE.md` 规则已约束）。

生成 Feature 文档：

prompt

```
1 /ai-native-prd 参考 @pm/epic/E00-core-business/README.md 生成 Feature
```

生成 User Story（最关键）：

prompt

```
1 /ai-native-prd 参考 @pm/epic/E00-core-business/F01-inventory-availability/ 生成 User Story
```

拆解 Feature 下的单独 User Story：

prompt

```
1 /ai-native-prd 参考 @pm/epic/E00-core-business/F01-inventory-availability/README.md 生成 "### 3. 按影片+门店查询已借出副本" 章节的 User Story
```

内置skill `.claude/skills/ai-native-prd/SKILL.md` （可根据需要定制）：

代码块

```
1 ---
2 name: ai-native-prd
```

description: 需求建模全流程—创建/完善 Epic、Feature、User Story 文档，以及校验和拆解。当用户要求创建需求文档、生成 Epic/Feature/User Story、校验 US 质量、拆解功能模块、梳理业务全景、规划里程碑时，必须使用此 skill。关键词：PRD、Epic、Feature、User Story、US、需求、用户故事、验收标准、数据契约、交互说明、最小粒度、拆解、功能模块、业务规则。

职责

覆盖 PM 需求建模的全部操作：Epic → Feature → User Story 三级文档的创建、完善、校验和拆解。

需求文档是 AI-Native SDLC 的源头—Epic 定义业务全景，Feature 拆解功能模块，User Story 直接驱动 UED 出设计稿、Dev 实现 API、QA 编写测试用例。

前置准备

执行任何操作前，先读取以下项目文件获取模板和规则：

- `pm/TEMPLATE.md` – Epic/Feature/User Story 完整模板
- `pm/CLAUDE.md` – 命名规则、必须章节、最小粒度标准

如用户提供了额外参考文档（如 PRD、已有 Epic/Feature），一并读取。

如用户未提供必要的上游文档，提示用户指定：

- 生成 Feature 时 → 提示用户提供 Epic README 路径
- 生成 User Story 时 → 提示用户提供 Feature README 路径
- 校验/拆解时 → 提示用户提供目标文件或目录路径

操作模式

根据用户意图自动选择对应模式。

模式 1: 生成 Epic

创建产品/系统级业务全景文档。

必须章节：

- 业务背景 · 业务目标 · 用户角色（角色+核心诉求表） · Feature 列表（ID/Feature/状态表，状态为 待拆解/已拆解/已完成） · 核心业务流程 · 约束与假设 · 里程碑（里程碑/包含 Feature/验收标准表）

输出：`pm/epic/Exx-name/README.md`（Exx 两位编号）

生成后提示用户审查 Feature 列表的完整性和优先级，建议使用 `/opsx:explore` 多轮脑暴发现遗漏。

42

43 **### 模式 2：生成 Feature**

44

45 从 Epic 拆解功能模块文档。

46

47 必须章节：

48 - 领域 · 概述 · 业务价值 · 用户故事概览 · 范围（In/Out of Scope） · 业务规则 · 关键能力（按 `### N. 能力名称` 组织） · 验收标准 · 依赖 · 技术备注（可选）

49

50 输出：`pm/epic/Exx-name/Fxx-name/README.md`（Fxx 两位编号），支持批量生成。

51

52 生成后提示用户审查关键能力完整性和 In/Out of Scope 边界，建议使用 `/opsx:explore` 多轮脑暴发现遗漏。

53

54 **### 模式 3：生成 User Story**

55

56 从 Feature 生成最小可交付单元文档。这是最关键的层级。

57

58 必须章节：

59 - 用户故事（作为...我需要...以便...） · 前置条件 · 验收标准（Given/When/Then） · 业务规则 · 数据契约（输入表/输出表/错误码表） · 交互说明 · 依赖 · openspec 流转 · 变更记录

60

61 生成要求：

62 - 验收标准覆盖正常和异常路径，QA 可据此直接输出测试用例

63 - 交互说明足够具体（页面布局、组件类型、交互行为、空状态/错误状态），UED 可据此直接输出设计稿

64 - 数据契约完整，字段类型、校验规则、必填项明确

65 - 满足最小粒度四项标准（见下方）

66

67 输出：`pm/epic/Exx-name/Fxx-name/USxx-name.md`（USxx 两位编号）

68

69 生成后提示用户进行人工校验，建议使用 `/opsx:explore` 多轮脑暴发现遗漏场景和边界条件。

70

71 **### 模式 4：校验 User Story**

72

73 对已有 US 文档逐项检查最小粒度四项标准：

74

75 | 标准 | 检查方法 |

76 | ----- | ----- |

4.3 人工校验（MUST）

概念说明

AI 生成的初稿虽然结构完整，但可能存在以下问题：

- 验收标准遗漏了关键异常场景
- 交互说明不够具体，UED 无法据此出设计稿

- 数据契约与实际数据库模型不一致
- User Story 粒度不合理（过粗需要继续拆分，过细需要合并）

因此，**PM 必须逐项审查**。可以借助 AI 辅助校验，但最终判断权在 PM。

具体操作

校验单个 User Story:

prompt

```
1 /ai-native-prd 校验 @pm/epic/E00-core-business/F01-inventory-availability/US01-check-inventory-availability.md
```

批量校验一个 Feature 下所有 US:

prompt

```
1 /ai-native-prd 校验 @pm/epic/E00-core-business/F01-inventory-availability/ 下所有US
```

检查是否遗漏 User Story:

prompt

```
1 /ai-native-prd 遗漏检查 @pm/epic/E00-core-business/F01-inventory-availability/
```

4.4 流转 to OpenSpec (MUST)

概念说明

人工校验通过后，User Story 需要流转 to OpenSpec 系统。OpenSpec 是 AI-Native SDLC 的 Spec 管理工具，将 User Story 转化为更细粒度的技术规格（specs），作为开发和测试的唯一事实源（Single Source of Truth）。

流转后会在 `openspec/changes/<change-name>/` 下自动生成：

- `proposal.md` — 变更提案（Why + What）

- `design.md` — 技术设计 (How)
- `specs/` — 最细粒度的需求规格说明
- `tasks.md` — 开发任务列表 (可选)

具体操作

方式 A：带上下文 `propose` (推荐)

当已有完整的 User Story 文档时，使用此方式确保 `specs` 与 US 验收标准完全对齐：

prompt

```
1 /opsx:propose 参考 @pm/epic/E00-core-business/F01-inventory-availability/US01-check-inventory-availability.md 生成提案，无需生成tasks
```

方式 B：直接 `propose` (适用于临时需求)

当需求比较明确、不需要先写正式 User Story 时，可以直接 `propose`：

代码块

```
1 /opsx:propose 开发对演员信息进行创建、查看、编辑、删除和搜索功能页面，以便维护完整准确的演员库并快速定位演员。
```

流转完成后，US01 文档中的 `## openspec 流转` 章节被同步更新：

代码块

```
1 ## 依赖
2 - **前置 US**：无
3 - **外部依赖**：`inventory` 表、`rental` 表、库存查询服务或数据库函数
4
5 ## openspec 流转
6 - **Specs name**：`inventory-availability-check`
7
8 ## 变更记录
9 | 日期 | 变更内容 | 受影响下游产物 |
10 |-----|-----|-----|
11 | 2026-04-15 | 初稿：新增单个库存副本可用性判断 US | openspec spec、后端查询接口、QA 测试用例 |
```

4.5 篝火评审 Specs (SHOULD)

概念说明

篝火评审是 AI-Native SDLC 的核心协作模式——PM + UED + Backend + Frontend + QA 多角色**并行**评审 OpenSpec 生成的 specs，确保：

- specs 与 User Story 验收标准完全对齐
- 交互细节满足 UED 出设计稿的要求
- 数据契约与数据库模型一致
- 验收标准可测试，异常场景覆盖充分

篝火评审的核心价值是**拉齐需求理解**，让所有角色在开始实施前就达成一致，避免后期反复澄清对齐。

具体操作

多角色视角评审（适用于单人项目）：

代码块

```
1  /ai-native-campfire 参考 @pm/epic/E00-core-business/F01-inventory-availability/US01-check-inventory-availability.md ，对 @openspec/changes/inventory-availability-check/ 进行评审
```

篝火评审后修正 specs：根据评审反馈内容，选择提议的修正方式。

5. 需求层级与模板

5.1 三级层级概览

代码块

```
1  Epic（产品/系统级）
2    |    业务全景、核心流程、里程碑规划
3    |    示例：E01 后台管理系统
4    |
5    └── Feature（功能模块级）
6        |    独立可交付的功能集，含业务规则和验收标准
7        |    示例：F02 演员管理
8        |
9        └── User Story（最小可交付用户价值）
```

- 10 拆到不可继续细分的最小粒度
- 11 直接驱动设计、开发和测试
- 12 对应 openspec change
- 13 示例：US01 演员信息增删改查

5.2 最小粒度四项标准

每个 User Story 必须同时满足以下四项标准，缺一不可：

标准	定义	检查方法
可独立验收	有明确的 Given/When/Then 验收标准	QA 可仅凭 US 文档编写测试用例
可独立设计	有完整的数据契约和交互说明	UED 可仅凭 US 文档输出设计稿
可独立实现	对应一个独立的前端页面或子功能， 以及与之对应的少数几个后端 API	Backend/Frontend 可独立开发和部署
AI 可消费	结构化字段完整，无歧义	AI 可基于 US 文档生成设计稿、API 代码和 测试用例

5.3 Epic 模板

存放位置： `pm/epic/Exx-name/README.md`

必须包含章节：

代码块

```
1  业务背景 · 业务目标 · 用户角色 · Feature 列表（含状态） · 核心业务流程 · 约束与假设 · 里程碑
```

模板结构：

代码块

```
1  # Exx: Epic 名称
2
3  ## 业务背景
4  （为什么需要这个 Epic，解决什么业务问题）
```

```
5
6  ## 业务目标
7    (可量化或可验证的目标)
8
9  ## 用户角色
10 | 角色 | 核心诉求 |
11 |-----|-----|
12 | ... | ... |
13
14 ## Feature 列表
15 | ID | Feature | 状态 |
16 |---|---|---|
17 | F01 | ... | 待拆解/已拆解/已完成 |
18
19 ## 核心业务流程
20    (主干流程描述或流程图)
21
22 ## 约束与假设
23 - (技术/业务/时间等约束条件)
24
25 ## 里程碑
26 | 里程碑 | 包含 Feature | 验收标准 |
27 |-----|-----|-----|
28 | M1 | ... | ... |
```

5.4 Feature 模板

存放位置: `pm/epic/Exx-name/Fxx-name/README.md`

必须包含章节:

代码块

```
1  领域 · 概述 · 业务价值 · 用户故事概览 · 范围 (In/Out of Scope) · 业务规则 · 关键能力
   · 验收标准 · 依赖 · 技术备注 (可选)
```

模板结构:

代码块

```
1  # Fxx: Feature 名称
```

```

2
3  ## 领域
4    (所属限界上下文)
5
6  ## 概述
7    (一句话描述功能)
8
9  ## 业务价值
10  - (价值点列表)
11
12  ## 用户故事概览
13  - (US 简要列表, 详见各 USxx 文档)
14
15  ## 范围
16  ### In Scope
17  - (功能范围)
18  ### Out of Scope
19  - (明确排除项)
20
21  ## 业务规则
22  1. (编号规则列表)
23
24  ## 关键能力
25  ### 1. 能力名称
26  - (能力描述)
27
28  ## 验收标准
29  - [ ] (Feature 级验收项)
30
31  ## 依赖
32  | 依赖 Feature | 依赖关系 |
33  |---|---|
34  | ... | ... |
35
36  ## 技术备注 (可选)
37  (实现注意事项)

```

5.5 User Story 模板

存放位置: `pm/epic/Exx-name/Fxx-name/USxx-name.md`

这是**最关键的层级**，结构化字段直接驱动下游角色的工作。

必须包含章节：

代码块

- 1 用户故事（作为...我需要...以便...） · 前置条件 · 验收标准（Given/When/Then） · 业务规则 · 数据契约（输入表/输出表/错误码表） · 交互说明 · 依赖 · openspec 流转 · 变更记录

模板结构：

代码块

```
1  # USxx: User Story 名称
2
3  ## 用户故事
4  作为 [角色], 我需要 [做什么], 以便 [获得什么价值]
5
6  ## 前置条件
7  - (执行此 US 前必须满足的条件)
8
9  ## 验收标准
10 - [ ] Given [前置条件] When [用户操作] Then [预期结果]
11 - [ ] Given [前置条件] When [异常操作] Then [错误处理]
12 - [ ] ...
13
14 ## 业务规则
15 1. (此 US 涉及的具体规则, 从 Feature 级细化而来)
16
17 ## 数据契约
18
19 ### 输入
20 | 字段 | 类型 | 必填 | 校验规则 | 说明 |
21 |-----|-----|-----|-----|-----|
22 | ... | ... | ... | ... | ... |
23
24 ### 输出
25 | 字段 | 类型 | 说明 |
26 |-----|-----|-----|
27 | ... | ... | ... |
28
29 ### 错误码
30 | 错误码 | 场景 | 说明 |
31 |-----|-----|-----|
32 | ... | ... | ... |
33
34 ## 交互说明
35 (界面行为描述: 触发条件 → 执行动作 → 结果反馈 → 异常处理)
36
```

```
37  ## 依赖
38  - **前置 US**：（必须先完成的 US）
39  - **外部依赖**：（API / 数据 / 服务）
40
41  ## openspec 流转
42  - **Specs name**：`<对应的 openspec specs 名称，可有多>`
43
44  ## 变更记录
45  | 日期 | 变更内容 | 受影响下游产物 |
46  |-----|-----|-----|
47  | ... | ... | ... |
```

6. 协作规则

6.1 与 UED 的对接点

对接时机	PM 提供	UED 消费	对接方式
User Story 完成后	交互说明章节	据此输出原型/设计稿	UED 读取 US 文档的交互说明
OpenSpec 流转后	specs 中的交互描述	据此输出高保真设计	篝火评审中对齐
评审反馈后	补充的交互细节	修正设计稿	PM 回写 US 文档并通知 UED

关键约束：User Story 的交互说明必须足够具体，使 UED 可仅凭该文档输出设计稿。需要包含：

- 页面布局（顶部/中部/底部分区）
- 组件类型（表格列定义、表单字段布局、按钮位置和样式）
- 交互行为（防抖、loading、校验时机）
- 空状态和错误状态的展示

6.2 与 Dev 的对接点

对接时机	PM 提供	Dev 消费	对接方式
OpenSpec 流转后	specs（含数据契约）	据此实现 API 和前端	Dev 读取 openspec/changes/ 下的 specs

实施中发现需求细节	及时补充 Spec	按修正后的 Spec 调整代码	Dev 通过 OpenSpec 反馈给 PM
开发完成后	—	/opsx:archive 归档	PM 验证并关闭 US

关键约束：

- 数据契约必须定义完整的输入/输出/错误码
- 代码必须来源于 Spec，如有偏差需及时回写

6.3 与 QA 的对接点

对接时机	PM 提供	QA 消费	对接方式
User Story 完成后	验收标准 (Given/When/Then)	据此编写测试用例	QA 读取 US 文档的验收标准
OpenSpec 流转后	specs 中的验收细节	据此补充自动化测试	篝火评审中对齐
实施完成后	—	执行验收测试	QA 按验收标准逐项验证

关键约束：

- 验收标准必须使用 Given/When/Then 格式
- 必须覆盖正常路径和异常路径
- QA 可仅凭 US 文档编写测试用例，无需额外澄清

7. 检查清单

以下是 PM 在 AI-Native SDLC 中必须遵守的 MUST 级要求，每完成一个 User Story 周期可对照勾选：

7.1 需求建模检查

- ☐ 需求已按 Epic → Feature → User Story 三级结构化建模
- ☐ User Story 已拆分到最小粒度，无法继续细分
- ☐ 每个 User Story 满足"可独立验收"标准：有 Given/When/Then 验收标准
- ☐ 每个 User Story 满足"可独立设计"标准：有完整的数据契约和交互说明
- ☐ 每个 User Story 满足"可独立实现"标准：对应独立页面/子功能 + 少数 API

- ☐ 每个 User Story 满足 "AI 可消费" 标准：字段完整、无歧义

7.2 文档质量检查

- ☐ Epic 文档包含全部必须章节（业务背景、业务目标、用户角色、Feature 列表、核心业务流程、约束与假设、里程碑）
- ☐ Feature 文档包含全部必须章节（领域、概述、业务价值、用户故事概览、范围、业务规则、关键能力、验收标准、依赖）
- ☐ User Story 文档包含全部必须章节（用户故事、前置条件、验收标准、业务规则、数据契约、交互说明、依赖、openspec 流转、变更记录）
- ☐ 所有需求文档使用中文，技术标识保持英文
- ☐ 文件命名符合规范（`Exx-name`、`Fxx-name`、`USxx-name`）

7.3 流程合规检查

- ☐ 使用 AI 生成了 Spec 初稿（MUST）
- ☐ AI 生成初稿后进行了人工校验（MUST）
- ☐ 校验通过的 User Story 已流转至 OpenSpec（`/opsx:propose`）
- ☐ OpenSpec 生成的 specs 覆盖了 US 文档中的所有验收标准
- ☐ US 文档的 `## openspec 流转` 章节已更新为实际的 change 信息
- ☐ 开发完成后已执行归档（`/opsx:archive`）

7.4 协作检查

- ☐ 交互说明足够具体，UED 可仅凭 US 文档输出设计稿
- ☐ 数据契约完整（输入/输出/错误码），Dev 可据此实现 API
- ☐ 验收标准覆盖正常和异常路径，QA 可据此编写测试用例
- ☐ 篝火评审已执行（SHOULD），各角色评审意见已收集和处理

8. 常见问题 FAQ

1. 什么时候用 `/opsx:explore`，什么时候直接写提示词生成文档？

A: ``/opsx:explore`` 适用于需求还比较模糊、需要梳理边界的场景，例如"我想做一个 XX 系统"。当需求已经比较明确（比如已有 Feature README），可以直接用提示词让 AI 按模板生成 User Story。

2. 一个 User Story 应该多大？

A: 一个 US 应该对应"一个独立的前端页面或子功能 + 少数几个后端 API"。如果一个 US 的验收标准超过 10 条，或者包含多个独立的界面交互流程，就需要继续拆分。以 Sakila 的 ``US01-actor-crud.md`` 为例，它包含列表/搜索/创建/编辑/删除/详情这些紧密相关的操作，共 17 条验收标准，但都围绕同一个"演员管理"页面，所以保持为一个 US 是合理的。

3. 篝火评审一定要做吗？

A: 篝火评审是 SHOULD 级别要求，推荐但不是强制。对于单人项目，可以在 Claude Code 中用 AI 模拟多角色评审。对于团队项目，篝火评审是拉齐需求理解的最有效方式。

4. User Story 和 OpenSpec specs 是什么关系？

A: User Story 是**需求描述**（业务视角，面向所有角色），OpenSpec specs 是**需求规格**（描述了你系统当前的行为，什么场景下应该具备什么表现）。一个 User Story 流转后会产生一到多个 specs。User Story 是 specs 的来源，specs 是 User Story 的细化。两者需要保持一致。

5. 需求变更怎么处理？

A:

1. 修改 User Story 文档，并提交git参考
2. 让AI对比提交变化（git diff），分析并更新相关specs
3. 通知受影响的下流角色（UED 修改设计稿、Dev 调整代码、QA 更新测试用例）

6. PM 需要了解技术细节吗？

A: 在 AI-Native SDLC 中，PM 不需要写代码，但需要理解基本的数据模型核心字段和 API 契约。这是因为 User Story 的数据契约字段需要 PM 定义，它直接影响后端 API 设计和前端数据展示。以 Sakila 为例，PM 需要知道 ``actor`` 有 ``first_name`` 和 ``last_name`` 字段，以便在数据契约中定义正确的校验规则。

7. 如何确保 AI 生成的文档质量？

A：三层保障：

- 1. **模板约束**：`pm/TEMPLATE.md` 定义了标准模板，AI 按模板生成确保结构完整
- 2. **CLAUDE.md 规则**：`pm/CLAUDE.md` 定义了 PM 角色的约束规则，AI 遵循这些规则确保内容规范
- 3. **人工校验**：AI 生成后 PM 必须逐项审查（MUST），这是质量的最终防线

8. 关键文件在哪里？

文件	说明
pm/CLAUDE.md	PM 角色的 Claude Code 约束规则
pm/TEMPLATE.md	Epic/Feature/User Story 完整模板
pm/sakila-prd.md	Sakila 产品全景文档
pm/epic/	需求三级目录结构
openspec/changes/	OpenSpec 进行中的变更
openspec/changes/archive/	已归档的变更
openspec/specs/	长期能力规格库